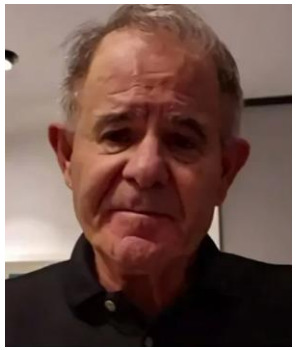
Clock Domain Crossing

Part 5

Amr Adel Mohammady

/amradelm

Save The Palestinian Children



Rami Igra
Former Israeli Intelligence Official

"Children in Gaza over 4 deserve to be starved"

Bezalel Smotrich
Israeli Minister of Finance

"Might be 'justified and moral' to cause 2 million Gazans to die of hunger, but world won't let us"

Ben Gvir
Israeli Minister of National Security

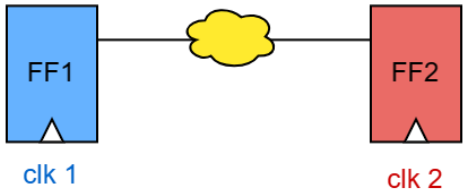
"Executing The Palestinian prisoner 'right solution to overcrowding'"

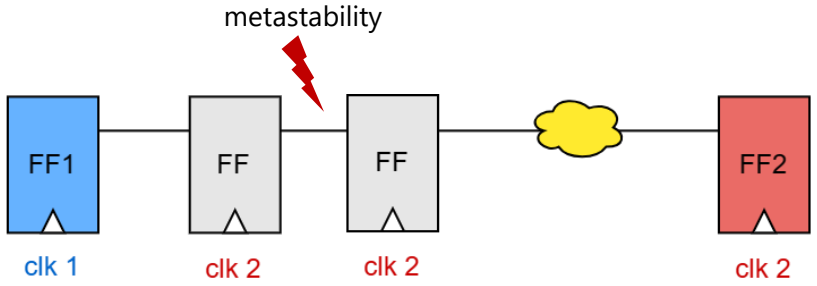
Yoav Gallant
Israeli Minister of Defense

"There will be no electricity, no food, no fuel, everything is closed, We are fighting human animals"

Introduction

- In the previous parts we saw how to send single bit across different clock domains.
- We also saw how to handle data duplication using pulse and edge synchronizers
- We then saw how to send multi bit signals using gray coding but that wasn't enough to send data values.
- **Our CDC concerns till now:**
  - Data corruption: Partially fixed. The system, till now, can only send multi-bit control signals but not data. Also, this data has a varying arrival time.
  - Data incoherence: Fixed. The metastable value settles within the synchronizers at 0 or 1 and then propagates to all the domain 2 blocks with the same settled value
  - Data loss: Fixed. The pulse is wide enough that it won't be missed by domain 2
  - Data duplication: Fixed if we use pulse/edge synchronizers
  - Chip burning: Fixed. We limited the metastability propagation between the synchronizers and reduced its occurrence frequency.
- In this part we will see how to send multi-bit data signal using the mux synchronization scheme





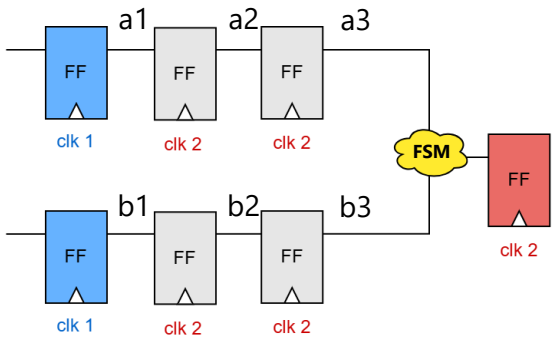
Reconvergence Issue

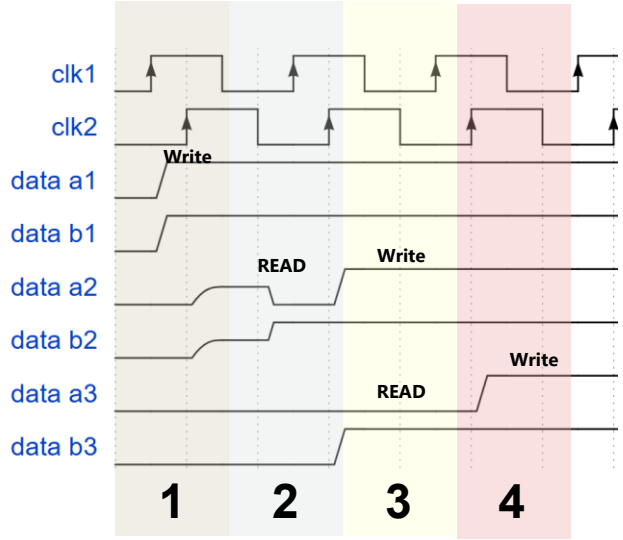
Multi Signal Reconvergence in the Receiving Domain

- When multiple signals are passed from the sending domain and then converge in the receiving domain, as shown in the diagram, we may get functional errors due to the difference in the settling time between the two signals.
- **Consider the example on the right:** Domain 1 sends a 2-bit control signal to domain 2.
  Initially we are in IDLE=2'b00

1. Domain 1 sends "2'b11 (WRITE)" to domain 2. The change occurs close to the edge of clk2 causing a metastability.
2. Signals a2 and b2 leave metastability and settle at different values "2'b10" (READ).
3. The value "2'b10" (READ) is passed to a3 and b3 and then to the combinational logic causing it to go to a (READ) state while the intended state was (WRITE).
4. The logic receives the correct value (WRITE) later but the damage is already done.

- This example shows the problem with sending multiple signals from one domain to another even with just 2 bits.
- **How to solve:**
  - Converge these signals in the sending domain then send them as one signal to the receiving domain. However, this is not always possible.
  - Use gray encoding to make sure only one signal changes at a time (Discussed before)
  - Use MUX synchronization scheme to pass these signals as a group (Will be discussed now)

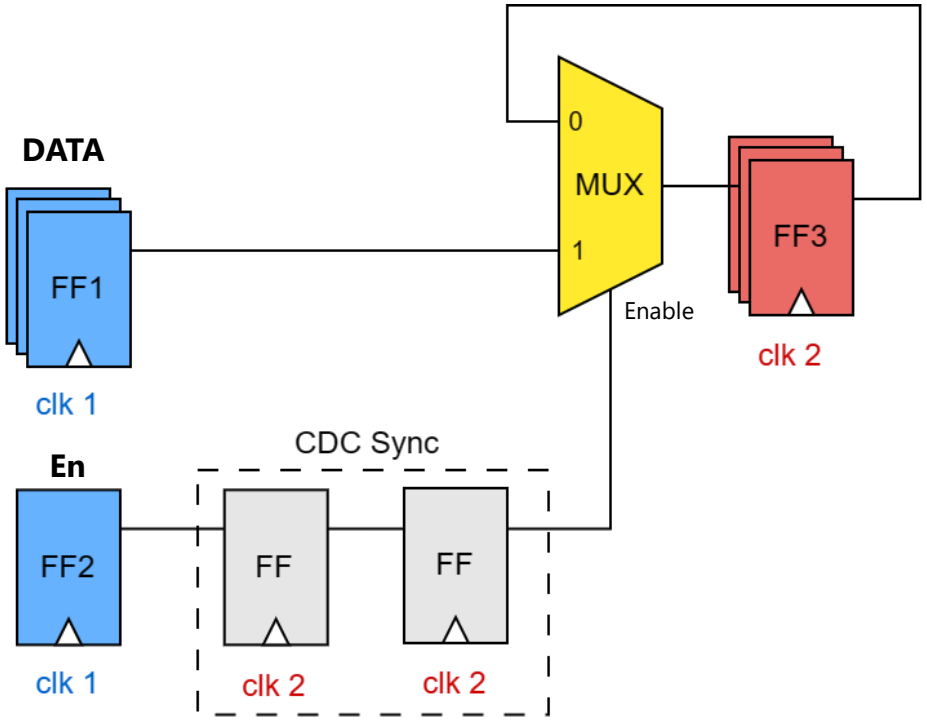




MUX Synchronizer

MUX Synchronizer

- The MUX synchronizer is one of the methods to pass multiple signals across clock domains.
- **It consists of two parts:**
  - The data part that goes directly to domain 2 without passing through synchronizers
  - An enable signal that pass through synchronizers then to domain 2 and is used to enable domain 2 to read the data of domain 1



MUX Synchronizer

- **The MUX Synchronizer works as follows:**

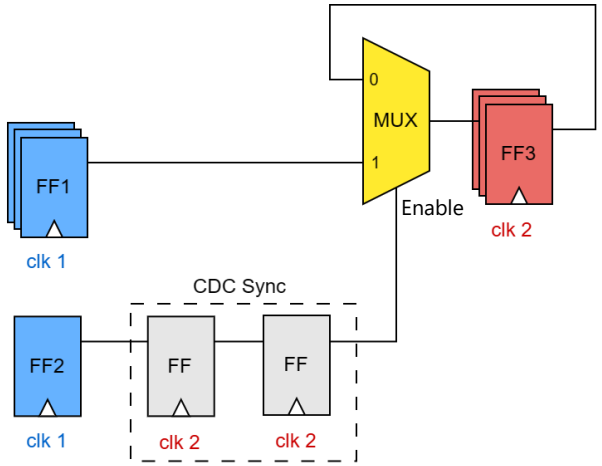
1. Domain 1 launches both the data and the enable signal to domain 2. The data goes directly to the MUX while the enable signal goes through the synchronizers
2. -The data reaches the MUX and waits for the select to be “1” to reach FF3.

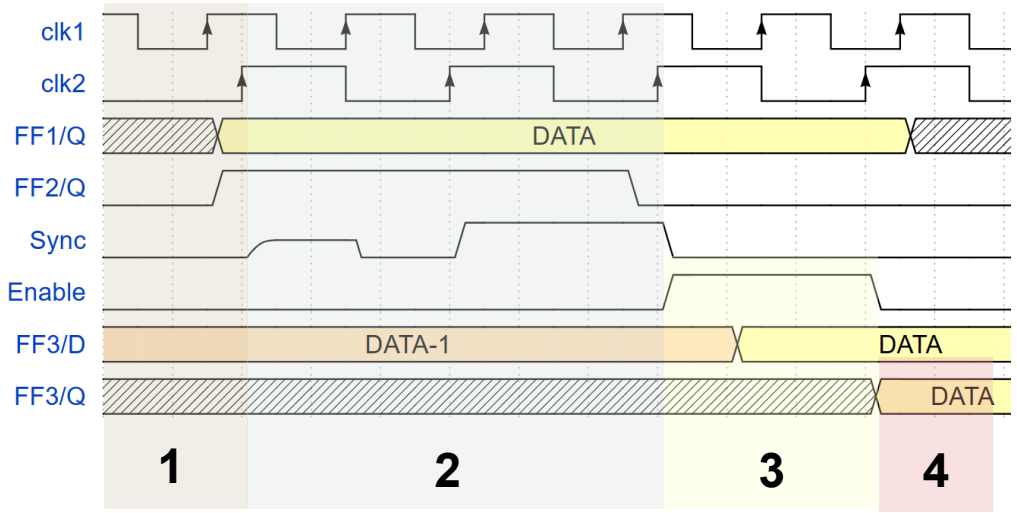
   -The MUX in front of FF3 still has select = “0”, so the D pin sees the old stored value inside FF3 and therefore FF3 won’t go metastable.

   -At the same time the enable signals goes through the syncs and cause metastability
3. -After some time the enable signal reaches domain 2 and then the MUX.

   -The data now propagates through the MUX to FF3/D.

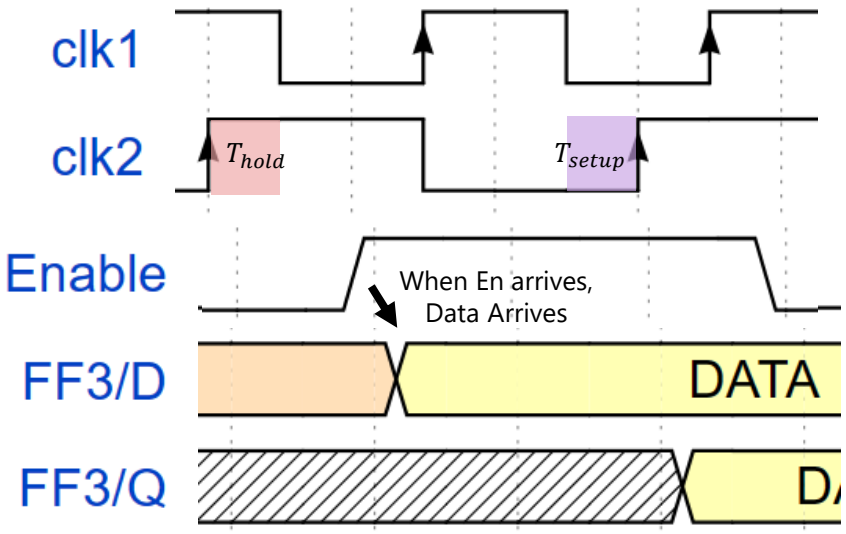
   -This propagation is controlled by the enable signal which is a synchronous signal within domain 2. This signal can be analyzed in STA to make sure FF3/D won’t change near a clock edge and won’t cause setup or hold violations.
4. The data is captured by FF3

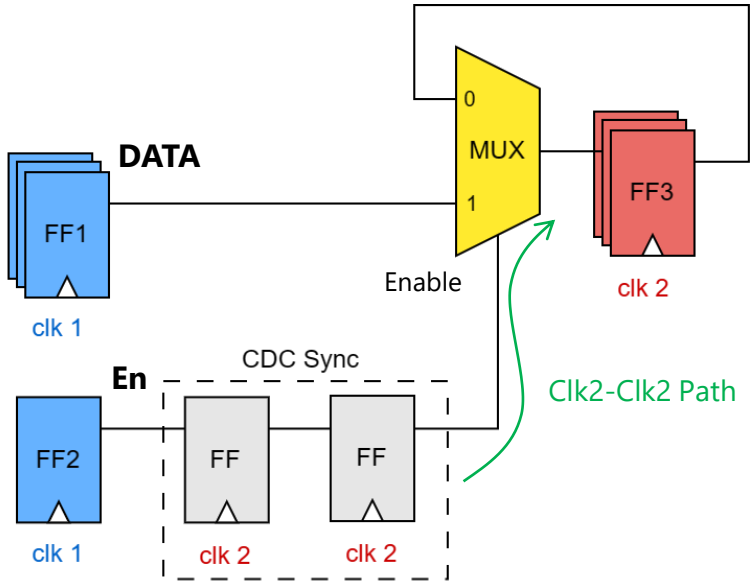




Why Does It Work?

- We are concerned that data from FF1 might reach FF3 during its setup/hold window, potentially causing metastability.
- **Lets ask: Will this happen under this structure?**
  - The enable signal acts as a gate for the data, preventing it from reaching FF3 until the enable signal arrives.
  - The enable signal is a timing path from clk2 to clk2, which can be analyzed using Static Timing Analysis (STA).
  - STA ensures that the enable signal does not arrive (the gate doesn't open) during the metastability window.
  - Therefore also the data will not arrive (pass the gate) during the metastability window.
- This needs an important assumption[1]: That by the time the enable reaches the MUX, the data should've arrived and is stable in front of the MUX

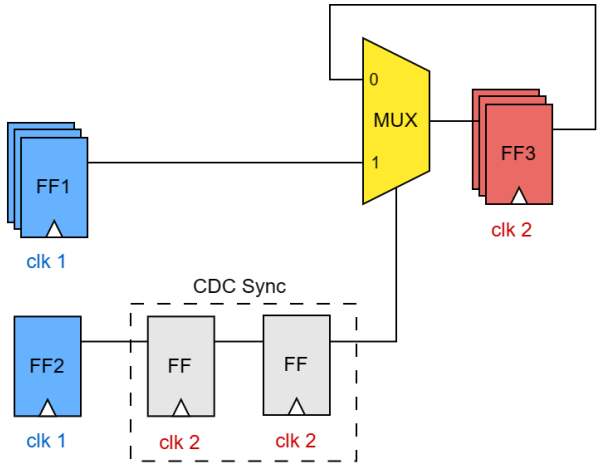


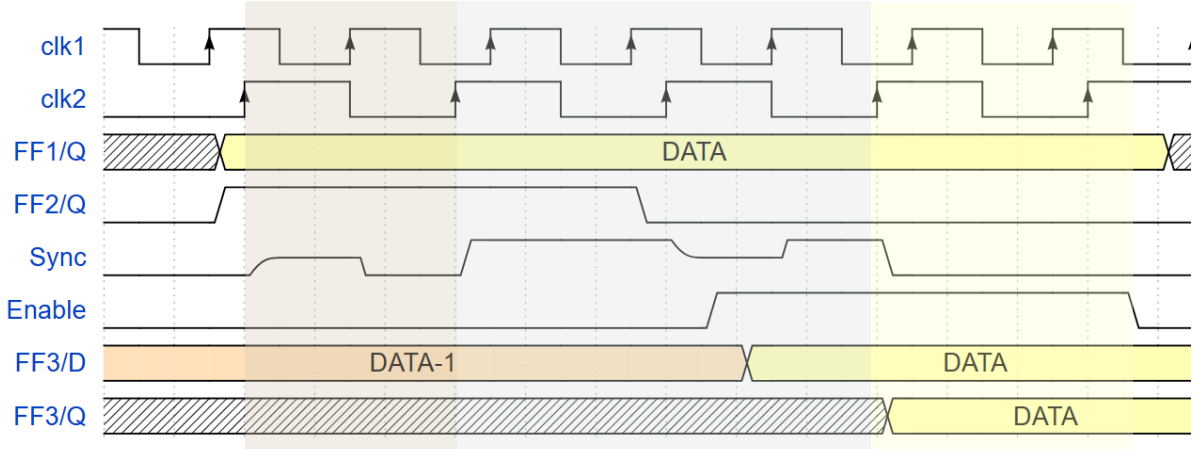


[1]: This assumption need to be enforced with a max delay constraint

MUX Synchronizer – Data Pulse Width

- **For how long do we need to hold data stable to ensure safe capture by the receiving domain?**
  - Most resources don't discuss this. Very few mention it's N+1 clk2 cycles where N is the number of sync FFs, but this can be shown to risk metastability[1]. The analysis below is my own and you are advised to take it with a grain of salt.
  - We have two concerns:
    1. We don't want the data to change before the enable signal arrives. Otherwise, the data won't be captured by domain 2 (The assumption from the previous slide)
    2. We don't want the data to change as long as the enable signal is high. Otherwise, the MUX will pass any asynchronous event from domain 1 to domain 2 and cause metastability.
  - To handle these concerns we need to consider the worst case:
    1. In the first clk2 cycle, the synchronizers goes metastable and settle at "0" instead of "1".
    2. After two (**N**) clk2 cycles, the enable becomes "1".
       At the same time, domain 1 lowers the enable signal, the change causes metastability and the synchronizers settle at "1" instead of "0".
    3. After one clk2 cycle + the delay from the sync to the MUX, the enable goes low "0". Any change in FF1/Q won't reach FF3/D
- We can assume[2] the sync->MUX delay = 0.5 clk2 cycle.
  So If we have **N** number of synchronizers then the data must be held for **N**+2.5 clk2 cycles



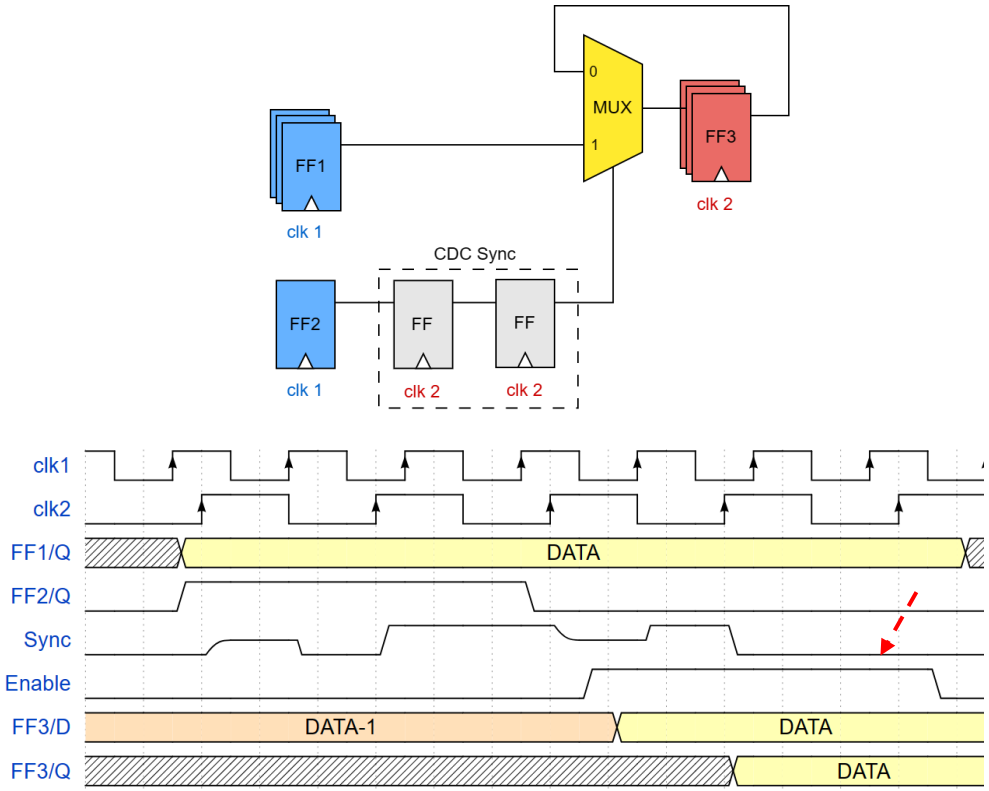


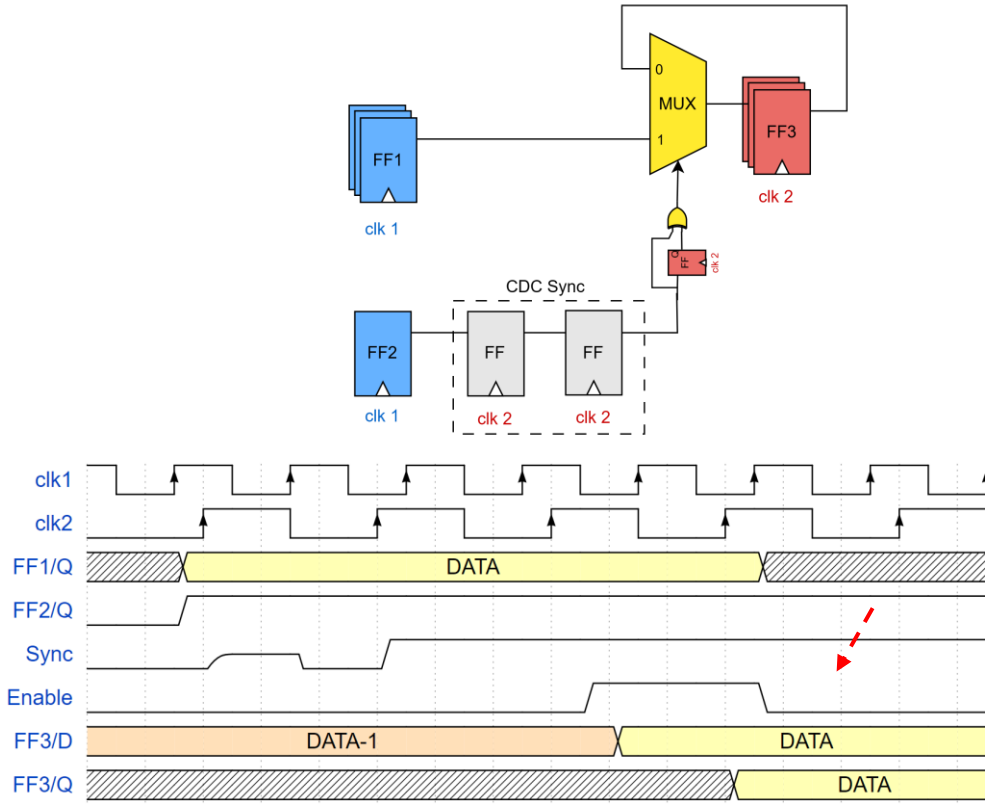
[1]: N+1 will work if we use pulse syntonizers. See next slide

[2]: This assumption need to be enforced with a max delay constraint

MUX Synchronizer With Pulse Synchronizer – Data Pulse Width

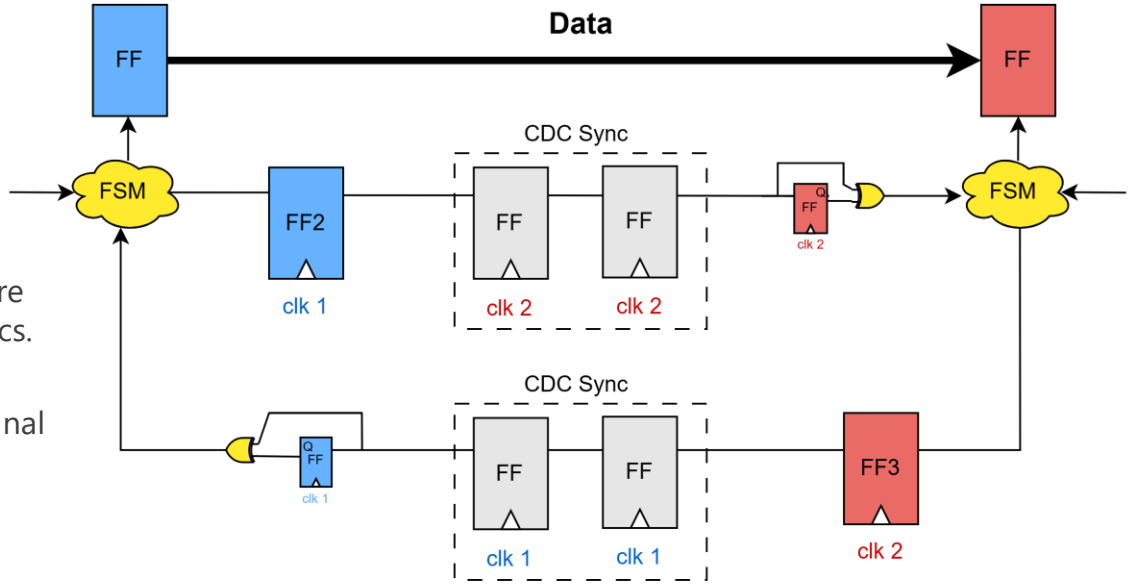
- Using a pulse synchronizer can shorten the requirement to N+1.5 cycles instead of N+2.5 cycles.
- This is because the pulse synchronizer will cause the enable to be active for one cycle instead of 2 (see red arrows).





Mux Synchronizers - Handshake

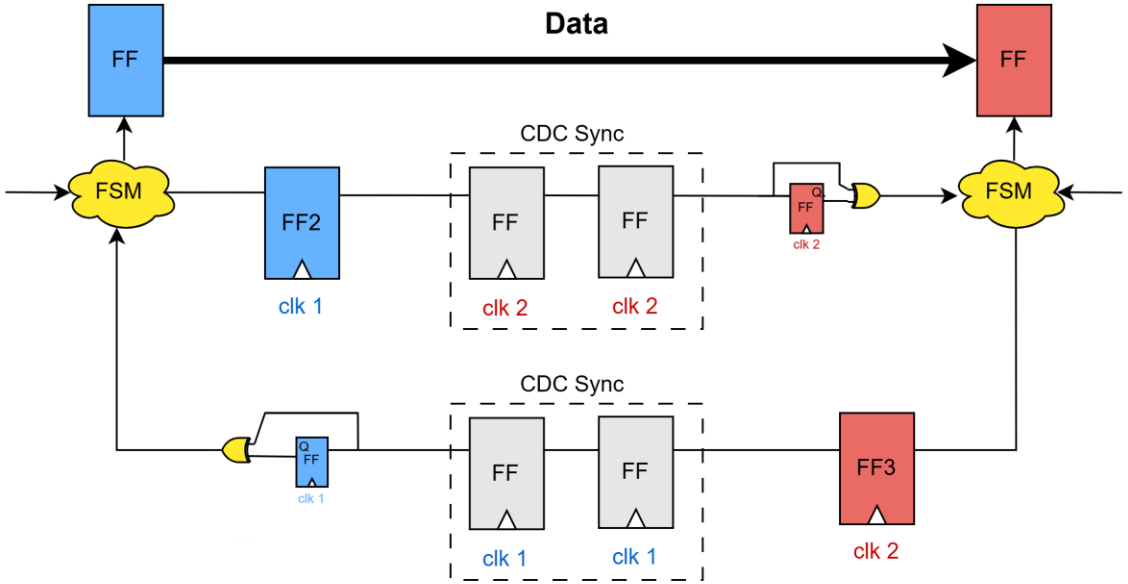
- Most designs combine the MUX synchronizer scheme with a handshake loop, where the sending domain keeps holding the data until the receiving domain responds with an acknowledgment signal that it has successfully captured the data.
- **This has several benefits over the previous approach:**
  - It ensures reliable data transfer. We only change the data when we receive a confirmation it was safely captured. So no need for the previous analysis
  - It allows the receiving domain to delay the transfer if it's not ready to accept it.
- **The operation goes as follows[1]:**
  1. Domain 1 sends a request/enable signal to domain 2 through FF syncs and at the same time it launches the data to domain 2.
  2. Domain 1 will keep holding the data and request signals until it receives an acknowledgment
  3. After a while, domain 2 receives the request. If it's ready to accept it, it will capture the data sent from domain 1 and send an acknowledgment back through FF syncs. If it's not ready it will keep the ack signal low.
  4. After a while, domain 1 receives the acknowledgment and lowers the request signal



[1]: You can watch an animation of this scheme here : https://lnkd.in/e3mb4D57

Mux Synchronizers – Disadvantage

- The main disadvantage of the MUX synchronizer scheme is that it doesn't allow domain 1 to send data every clock cycle
- Instead, we hold the data stable for some time waiting for it to be captured and also waiting for an ack signal.
- This latency makes MUX schemes not suitable for designs that require high-speed processing.
- In the next part we will study another CDC method that can handle high-speed requirements.



References

1) http://www.sunburst-design.com/papers/CummingsSNUG2008Boston_CDC.pdf
2) https://www.uio.no/studier/emner/matnat/ifi/IN3160/v21/timeplan/in3160-l92-clock-domains.pdf
3) https://ieeexplore.ieee.org/document/1676187
4) https://www.edn.com/keep-metastability-from-killing-your-digital-design/
5) https://people.ece.ubc.ca/~edc/7660.jan2018/lec11.pdf
6) https://www.onsemi.com/pub/Collateral/AN1504-D.PDF

Thank You!